

Turborepo POS Project - Work Report

1. Learning Turborepo

Turborepo is a build system used to manage a monorepo, which is a single repository that holds more than one project. In our case, we have two POS systems: one for a Pakistan electric store and one for a Dubai grocery store. The two apps share common code through shared packages, instead of duplicating the same code in two places. Turborepo also speeds up builds with smart caching, meaning it only rebuilds the part of the project that actually changed.

2. Setting Up the Turborepo

We created the monorepo using the create-turbo command. The default app folders, which were named web and docs, were renamed to pos-pakistan and pos-dubai to match our two clients. We also fixed package manager issues along the way, including a corrupted node_modules folder and missing dependencies, by clearing the cache and reinstalling everything from a clean state.

3. Building the Shared Database Package

We created a shared package called packages/database. Inside it, we set up two separate Prisma schemas, one for Pakistan and one for Dubai, since each store sells different kinds of products. The Pakistan schema includes fields like voltage and warranty months, while the Dubai schema includes fields like expiry date and weight in grams. Even though the schemas are different, both live inside the same shared package, which keeps the database setup organized in one place.

4. Creating the Databases

Using Prisma migrations, we generated two separate SQLite database files, one for each store. This means Pakistan and Dubai each have their own real, independent database, even though the underlying tools and structure are shared. This matches how real businesses usually keep each client's data separate while reusing the same backend technology.

5. Connecting the Apps to the Database

Each app, pos-pakistan and pos-dubai, was connected to the shared database package by adding it as a dependency. Environment variables were set up in each app to point to its own database file. We also installed and configured Tailwind CSS in the Pakistan app, since the default Turborepo template did not include it.

6. Building CRUD for Products

CRUD stands for Create, Read, Update, and Delete, and this is the core functionality we built for managing products. We created a products list page to view all products, an add product page to create new ones, an edit page to update existing products, and a delete action to remove them. Server Actions were used for all of these operations, since they allow the app to handle form submissions directly on the server without creating separate API endpoints.

7. Fixing Issues Along the Way

Several real issues came up during development and were fixed one by one. These included a missing dependency causing the app to crash on startup, an incorrect file name breaking an import, missing environment variables causing database connection errors, and an outdated way of reading page parameters that needed to be updated for the newer version of Next.js. Each issue was identified from the error message and corrected directly in the code.

8. Where We Are Now

By the end of this work session, both the Pakistan and Dubai apps were running successfully with their own databases. The product list, add product, and edit product pages were working for both stores. The next steps going forward will be polishing the styling and eventually building the staff-facing checkout screen that will use this same product data.